1. Values, Variables, Types, and Zero Values

01. Values, Variables, Types, and Zero Values

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `01-values-variables-types`

Learning objectives

- Declare variables with var and short declaration
- List basic types and their zero values
- Explain why every variable always has a defined value in Go
- Choose between typed and untyped constants

Prerequisites

Complete prior numbered lessons. Type every example yourself.

Values and types

Every value in Go has a **type**. Types answer: what operations are allowed? What size does it occupy? What is its zero value?

Common types: - Integers: `int`, `int64`, `uint`, `byte` (alias for `uint8`) - Floats: `float64` (prefer over `float32` unless you have a reason) - Boolean: `bool` (`true` / `false`) - String: `string` (immutable sequence of bytes, usually UTF-8)

```
var age int = 30
var name string = "Ada"
ready := true          // short declaration; type inferred
const MaxRetries = 3   // untyped constant unless given a type
```

Zero values

If you declare a variable without assigning, Go sets the **zero value**: - numbers → `0` - bool → `false` - string → `""` - pointers/slices/maps/channels/functions/interfaces → `nil`

This removes a whole class of “undefined variable” bugs common in other languages.

```
var n int
fmt.Println(n) // 0, always safe to read
```

Memory intuition

A variable name is a label for a location in memory holding a value of a known type. `int` on a 64-bit machine is typically 8 bytes. You do not manage that manually — but knowing sizes helps later with slices, structs, and performance.

Common mistakes (especially from Python instincts)

- Using `None`-like thinking: Go has `nil` only for reference types, not for ints/strings

- Shadowing variables with `:=` inside inner blocks accidentally
- Assuming uninitialized variables are dangerous — in Go they are zeroed

Exercises

Work in `exercises/01-values-variables-types/`. Read that folder's `README.md`.

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/01-values-variables-types/` until you have an honest failing attempt.

Self-check

1. What is the zero value of `string`? Of `*int`?
2. When must you use `var` instead of `:=`?
3. Why are zero values useful in APIs?

Further reading

- https://go.dev/ref/spec#The_zero_value
- https://go.dev/tour/basics